MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

STUDENT INTERNSHIP PROGRAM

REPORT ON

Virtual Internship Programme

Internship/Certificate Provider

Cisco Networking Academy

Submitted by-
Name – Kadam Om Shivnath
PRN – 202501040064
Exam Seat No: 41
Internship Duration: 8 weeks

Department of Computer Engineering

MIT Academy of Engineering, Alandi(D)

Pune Maharashtra - 412105

Academic Year 2026-27

MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

CERTIFICATE

This is to certify that the "Student Internship Program (SIP)" report submitted by **Om Shivnath Kadam, PRN-** 202501040064 is work done by him and is submitted during 2026-2027 academic year.

Internship Guide

Prof. Disha Sengupta

Internship Coordinator

(Dr. Yogesh B. Pawar / Mr. Jalaj limaye)

HOD

Department of Computer Engineering

(Dr. Pramod Ganjewar)

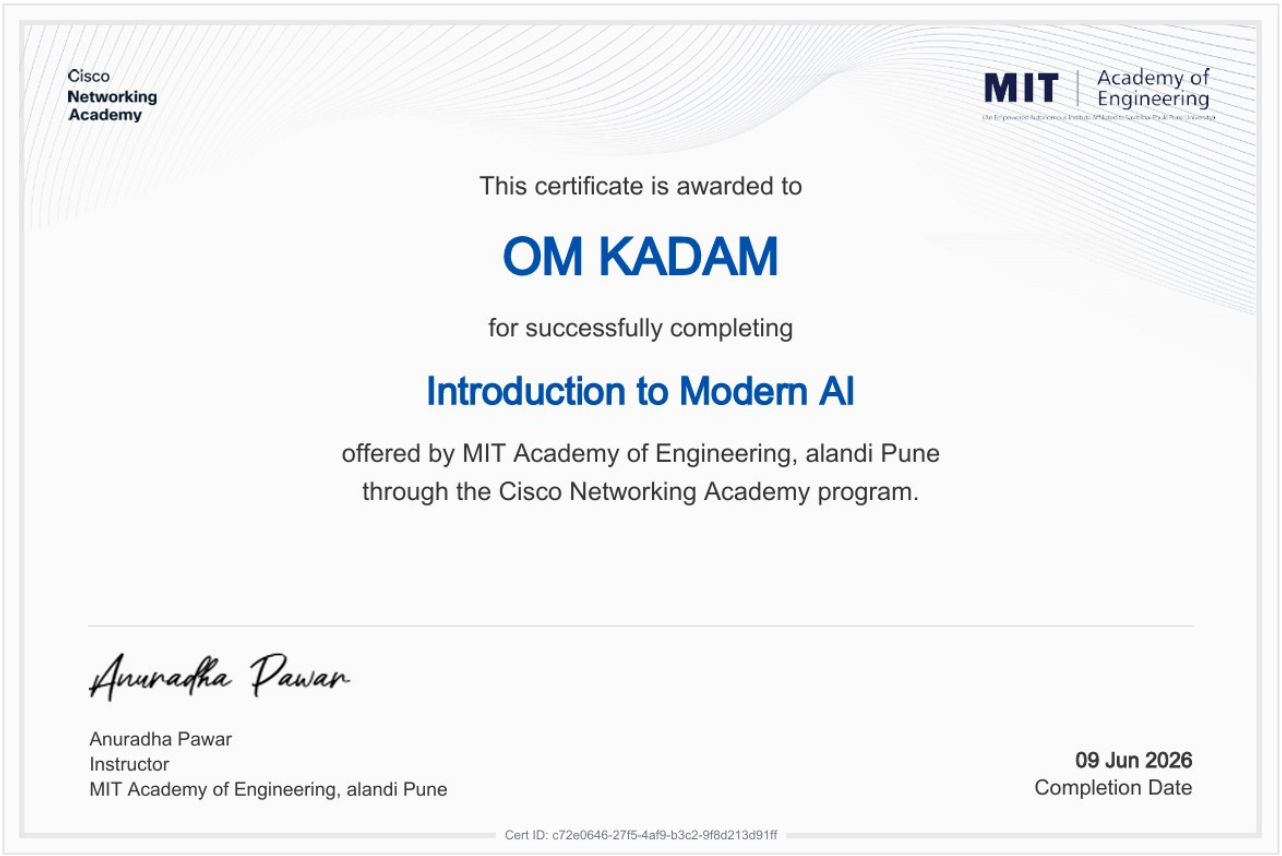
Department of Computer Engineering

MIT Academy of Engineering, Alandi(D)

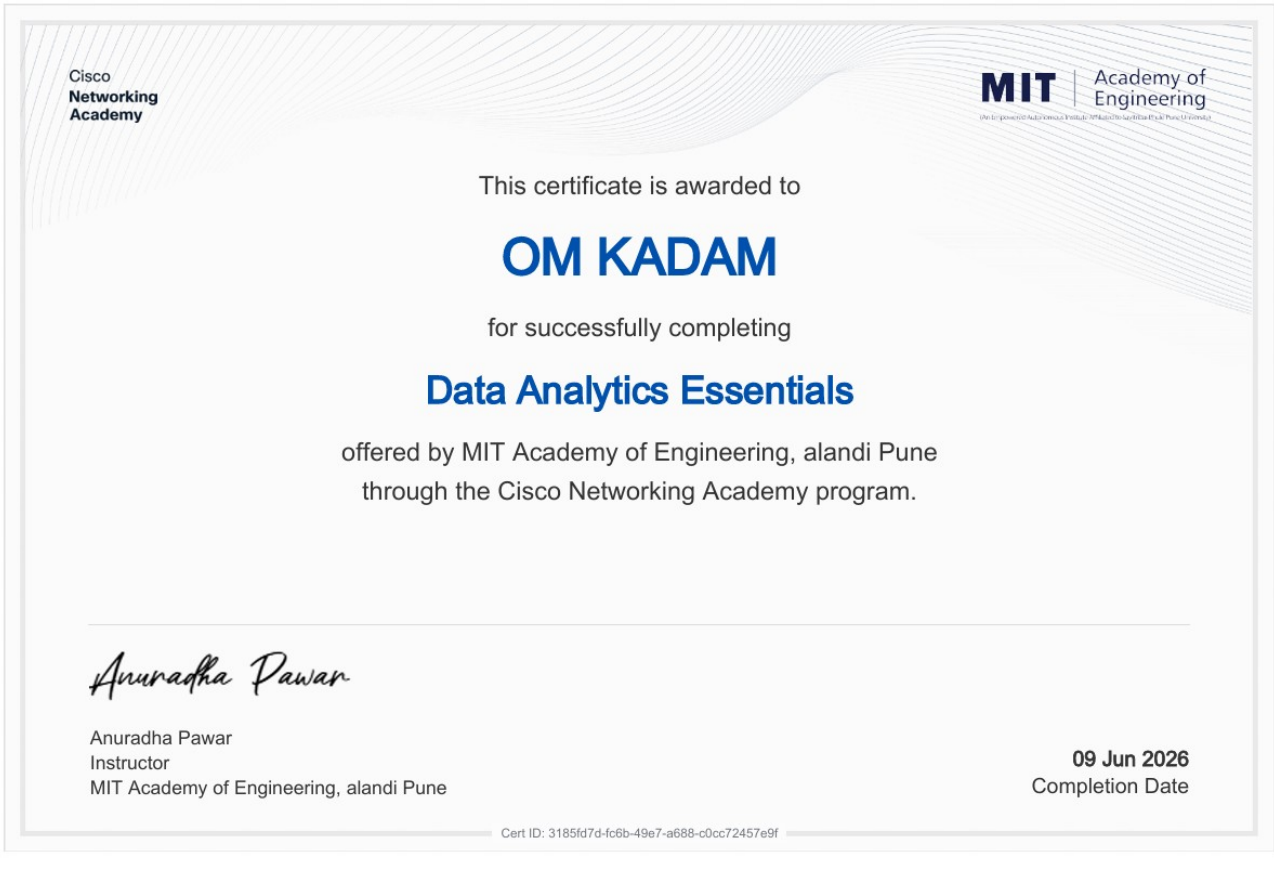
Pune Maharashtra – 412105

Certificates

1. **Introduction to Modern AI:**



2.Data Analyatics Essentials



3. Apply AI: Analyze Customer Reviews:



Cisco Networking Academy

MIT | Academy of Engineering

This certificate is awarded to

OM KADAM

for successfully completing

Apply AI: Analyze Customer Reviews

offered by MIT Academy of Engineering, alandi Pune
through the Cisco Networking Academy program.

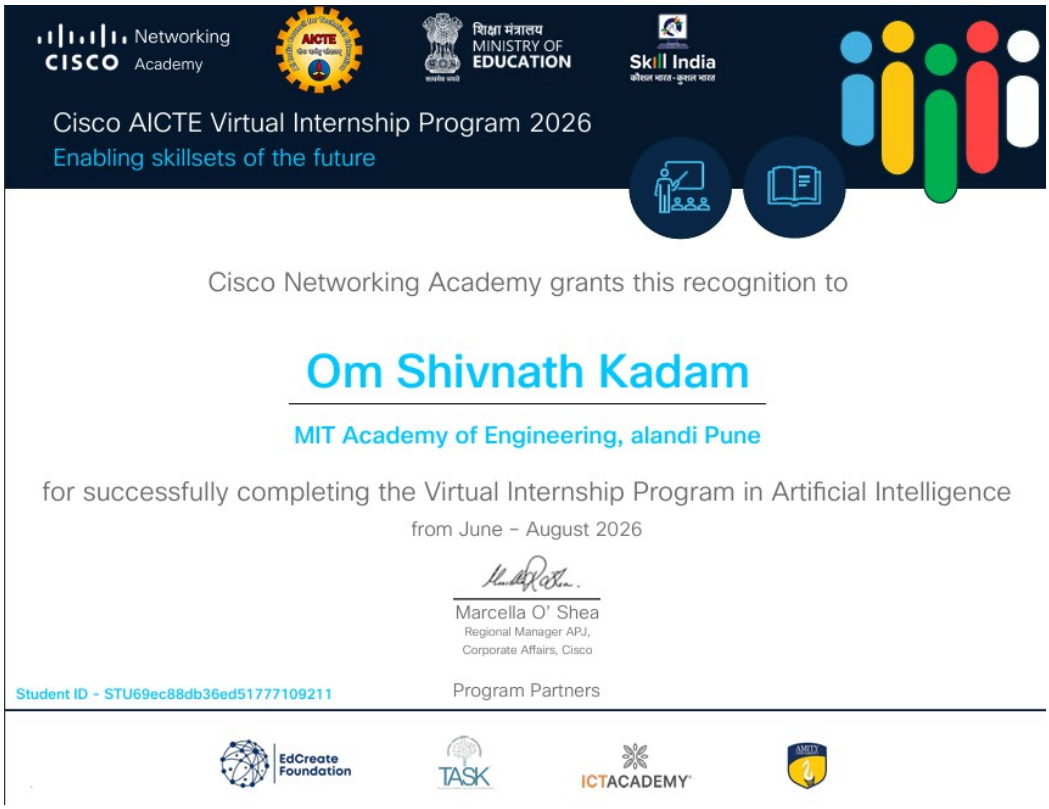
Anuradha Pawar

Anuradha Pawar
Instructor
MIT Academy of Engineering, alandi Pune

09 Jun 2026
Completion Date

Cert ID: 24ccaba6-2c5a-4994-8d01-b591440e2500

4. Project Completion Certificate:



Networking Academy | AICTE | शिक्षा मंत्रालय MINISTRY OF EDUCATION | Skill India

Cisco AICTE Virtual Internship Program 2026
Enabling skillsets of the future

Cisco Networking Academy grants this recognition to

Om Shivnath Kadam

MIT Academy of Engineering, alandi Pune

for successfully completing the Virtual Internship Program in Artificial Intelligence
from June - August 2026

Marcella O' Shea
Regional Manager APJ,
Corporate Affairs, Cisco

Student ID - STU69ec88db36ed51777109211

Program Partners

EdCreate Foundation | TASK | ICTACADEMY

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **MIT Academy of Engineering, Alandi (D)** for providing me with the opportunity to undertake the **Student Internship Program (SIP)**, which has significantly enhanced my technical knowledge and practical skills.

I am thankful to **Cisco Networking Academy** for organizing the **Virtual Internship Programme** and providing excellent learning resources, hands-on training, and guidance throughout the internship. The knowledge gained during this internship has strengthened my understanding of Artificial Intelligence, Cloud Computing, and their real-world applications.

I express my heartfelt gratitude to my **Internship Guide, Disha Sengupta** for his valuable guidance, continuous encouragement, and constructive suggestions throughout the internship and while preparing this report.

I would also like to thank **Dr. Yogesh B. Pawar and Mr. Jalaj Limaye Department Internship Coordinator**, for her constant support, motivation, and coordination during the internship program.

I extend my sincere thanks to **Dr. Pramod Ganjewar, Head of the Department of Computer Engineering**, for providing the necessary facilities and encouragement to complete this internship successfully.

Finally, I would like to thank all the faculty members, trainers, and everyone who directly or indirectly contributed to the successful completion of this internship and this report.

Om Shivnath Kadam
PRN: 202501040064

TABLE OF CONTENTS

Section No.	Title	Page No.
1	**INTRODUCTION**	**8**
1.1	Objective and Scope of Internship / Project	9
	Primary Objectives	9
	Scope	10
	Limitations	10
1.2	Organization / Institute Profile	11
1.3	Background of Project Undertaken	11
2	**PROJECT DESCRIPTION**	**13**
2.1	Description of Tasks and Assignments	13
2.1.1	Requirement Analysis	13
2.1.2	Case Dataset	14
2.1.3	Supported Troubleshooting Categories	15
2.1.4	Deterministic Python Rule Checker	15
2.1.5	AI Diagnosis Engine	16
2.1.6	Human Review Workflow	16
2.1.7	Dashboard and Operational Analytics	17
2.1.8	Responsible AI Considerations	17
2.1.9	End-to-End Workflow	18
2.2	Tools, Technologies and Methods Used	19
2.3	Challenges Faced and Solutions Provided	20
2.4	Outcome of Internship / Certification	21
3	**LEARNING AND OBSERVATIONS**	**21**
3.1	Technical Skills Acquired	21
	Networking Fundamentals	21
	Cisco Command Interpretation	22
	Python Development	23
	AI Prompt Design	23
	Data Handling and Visualization	23
	Testing and Debugging	23
3.2	Professional / Soft Skills Developed	23
4	**RESULTS AND DISCUSSIONS**	**25**
4.1	Key Findings or Results	25
	Suggested Test Procedure	25
4.2	Data Analysis	25
4.3	Interpretation of Results	26
4.4	Contribution to the Organization / Academic Work	27
5	**CONCLUSION**	**28**
5.1	Future Enhancements	28
6	**REFERENCES**	**30**
	APPENDIX E: PLAGIARISM / SIMILARITY REPORT	**32**

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Supported Network Troubleshooting Categories and Diagnostic Evidence	15
Table 2	Tools, Technologies, and Engineering Methods Used	19
Table 3	Potential System Challenges and Recommended Handling Approaches	20
Table 4	Key Cisco IOS Diagnostic Commands and Inspection Roles	22
Table 5	Data Analysis, Performance Metrics, and Review Evaluation Framework	26

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1	Conceptual Architecture of the NetSage AI Diagnostic System	13
Figure 2	NetSage AI Case Explorer Telemetry Inspection Interface	14
Figure 3	Operational Analytics Dashboard and Diagnostic Distribution	17
Figure 4	End-to-End Diagnostic and Human-in-the-Loop Review Workflow	19

1. Introduction

Computer networks connect computers, servers, applications, and other devices so that they can exchange information reliably across distributed environments [1]. Reliable connectivity is essential in educational laboratories, corporate enterprises, and mission-critical cloud services. When an enterprise network ceases to operate as expected, network technicians and systems engineers must rapidly identify the root cause before applying remedial configurations. A disciplined, correct diagnosis expedites the restoration of network services while preventing spurious and potentially destabilizing alterations to active switching and routing topologies.

Network troubleshooting is inherently an iterative, multi-stage diagnostic process. A user or client application may report symptoms such as the inability to open an intranet portal, an endpoint failing to reach a remote IP subnet, or intermittent layer-3 reachability. However, the observable symptom alone does not unambiguously determine the underlying fault. For instance, the inability to establish a connection to an application server may stem from an invalid IP configuration, an unroutable default gateway, mismatched Virtual Local Area Network (VLAN) assignments, an inactive 802.1Q trunk, an erroneous routing table entry, a filtering Access Control List (ACL), a misconfigured DNS resolver, or an unresponsive server host. Diverse underlying faults frequently manifest identical end-to-end failure symptoms across intermediate nodes.

Industry-standard Cisco Internetwork Operating System (IOS) commands offer authoritative diagnostic telemetry regarding device operating status, port states, and control-plane configurations [2], [3]. Diagnostic commands such as `show ip interface brief`, `show ip route`, `show vlan brief`, and `show interfaces trunk` empower practitioners to inspect physical interfaces, operational routing tables, Layer 2 broadcast domains, and inter-switch trunk links. Nevertheless, students and junior network technicians frequently encounter cognitive overload when attempting to select the most appropriate command to execute next and when synthesizing disparate pieces of diagnostic evidence. Learners often jump to premature conclusions before gathering sufficient telemetry to rule out competing hypotheses.

NetSage AI is proposed as an intelligent educational and diagnostic assistant designed for troubleshooting simulated Cisco enterprise network environments [4]. The system

systematically couples deterministic Python-based rule validation with Large Language Model (LLM) reasoning [5] to interpret complex command outputs and correlate observable symptoms. The resulting output provides a structured, explainable diagnostic profile comprising the primary root cause hypothesis, applicable OSI layer, confidence indicator, supporting empirical evidence, next recommended diagnostic command, and step-by-step remediation procedures. Crucially, the system enforces a strict human-in-the-loop paradigm: an authenticated human reviewer retains ultimate authority to accept, modify, or reject any AI-generated hypothesis.

The proposed architecture strictly maintains that AI diagnostic predictions must be handled as probabilistic hypotheses rather than verified ground truth. Under no operational circumstance is the assistant permitted to execute automated configuration adjustments directly onto physical or simulated networking infrastructure. Any recommended corrective command must undergo rigorous human review, followed by manual deployment in an authorized simulation environment and empirical verification through targeted post-remediation tests. This design guarantees educational efficacy, interpretability, and operational safety throughout the troubleshooting lifecycle.

1.1. Objective and Scope of Internship / Project

The overarching objective of this research is to conceptualize, engineer, and validate an AI-assisted network troubleshooting application that enables students and technical operators to organize diagnostic evidence, systematically isolate networking faults, and develop repeatable troubleshooting methodologies while preserving human agency over configuration changes.

Primary Objectives

- Ingest heterogeneous network problem symptoms, user topology notes, and raw Cisco IOS CLI command outputs [2].
- Execute deterministic Python-based rule algorithms to evaluate boundary conditions and detect deterministic configuration anomalies.
- Synthesize structured, evidence-grounded AI diagnoses that strictly delineate empirical observations from speculative hypotheses [5].
- Recommend targeted follow-up diagnostic commands and validated manual remediation steps.

- Provide a dedicated human-in-the-loop review workflow featuring Accept, Edit, and Reject decision mechanisms.
- Maintain separate, immutable audit trails for raw AI diagnostic proposals and final human-adjudicated review decisions.
- Provide real-time operational analytics and aggregate case distribution metrics through an interactive graphical dashboard [6], [7].

Scope

The primary operational scope of NetSage AI encompasses educational networking laboratories, particularly Cisco Packet Tracer simulation environments [4] and junior network engineering training scenarios. The application is tailored to assist students and trainee administrators in navigating complex telemetry; it is not designed to replace certified network engineers or autonomously manage enterprise production fabrics. The troubleshooting domains addressed include Layer 2 VLAN configurations, default gateways, DHCP leases, DNS resolution, static and dynamic IP routing, Access Control Lists (ACLs), Network Address Translation (NAT), and basic 802.11 wireless connectivity.

The formal project design specifies a comprehensive evaluation repository containing a minimum of 30 curated troubleshooting test cases, structured system prompt templates, a deterministic rule checker, interactive dashboard analytics, and a responsible-AI evaluation log with at least five documented human-corrected diagnoses. These targets establish the baseline acceptance criteria for experimental validation.

Limitations

- Diagnostic fidelity is directly contingent upon the completeness, accuracy, and relevance of the diagnostic command outputs supplied by the user.
- The confidence indicators generated by the AI reasoning engine reflect heuristic model estimations and do not constitute formal mathematical certainty.
- Deterministic rule checks can only identify fault conditions that are explicitly captured within the provided CLI telemetry and encoded into the rule definitions.
- Automated or unsupervised configuration changes are strictly prohibited to ensure operational safety and prevent network disruption.
- All diagnostic performance metrics, agreement rates, and accuracy evaluations must be derived from empirical test datasets rather than subjective conjecture.

1.2. Organization / Institute Profile

MIT Academy of Engineering (MIT AOE), located in Alandi (D), Pune, Maharashtra, is an autonomous academic engineering institution affiliated with Savitribai Phule Pune University [8]. Operating under the Department of Computer Engineering, the institution fosters advanced undergraduate and postgraduate research in intelligent computing, distributed systems, and applied artificial intelligence. The research proposal and experimental evaluation documented herein have been developed within this institutional framework in accordance with national technical education internship guidelines [9].

The project was conducted under the auspices of the Cisco AICTE Virtual Internship Programme in Artificial Intelligence, facilitated by Cisco Networking Academy in partnership with the All India Council for Technical Education (AICTE) [9]. If specific corporate host credentials, external organizational profiles, or formal training registrations are required, they shall be incorporated into the final institutional registry upon formal verification.

1.3. Background of Project Undertaken

In modern computer networking curricula, an enduring pedagogical challenge is the disconnect between memorizing discrete CLI configuration commands and applying those commands to resolve complex, interdependent network failures [3]. A student may know the exact syntax required to display an IP routing table via `show ip route` [2] yet lack the systemic reasoning required to determine whether an observed routing anomaly explains an end-to-end transport failure. Effective troubleshooting demands synthesizing multi-layer telemetry encompassing device port states, Layer 2 broadcast segregation, Layer 3 routing reachability, and transport-layer access control policies [1].

As an illustrative scenario, consider an endpoint workstation configured via DHCP that successfully pings its local default gateway router interface but fails to establish a TCP session with an internal enterprise server situated in VLAN 30. While successful communication with the default gateway validates local physical link integrity and Layer 3 gateway reachability, it leaves numerous plausible failure modes unexamined. The failure may arise from an unadvertised route to VLAN 30 on an intermediate switch, an 802.1Q trunk link omitting VLAN 30 from its allowed list, an inbound ACL discarding traffic on the destination router interface, or an unassigned switch access port. Discriminating between these potential root causes requires executing and interpreting an ordered sequence of

diagnostic commands, including `show ip route`, `show access-lists`, `show interfaces trunk`, `show vlan brief`, and `show ip interface brief` [2].

NetSage AI addresses this pedagogical challenge by embedding structured, disciplined reasoning into the learning workflow. By orchestrating deterministic rule checks and explainable AI inference, the system prompts learners to ask targeted diagnostic questions, analyze empirical device telemetry, and retain full technical responsibility for all remediation decisions.

2. Project Description

NetSage AI is engineered as an intelligent diagnostic assistant tailored for Cisco networking laboratories [4]. The architecture decouples analytical responsibilities into two distinct, complementary subsystems: a deterministic Python rule-checking engine [10] and an evidence-grounded AI reasoning engine [5]. Deterministic rules parse explicit syntactic indicators and identify non-negotiable configuration errors (such as mismatched subnet masks or shutdown interfaces), whereas the AI reasoning engine synthesizes free-form problem symptoms with structured telemetry to generate contextual root cause hypotheses.

The generated diagnostic output is delivered in a highly structured schema featuring seven core analytical attributes: the primary root cause hypothesis, associated OSI layer, qualitative confidence classification (High, Medium, or Low), citations of empirical CLI evidence, next recommended diagnostic command, step-by-step remediation procedure, and a transparent reasoning narrative. The human operator reviews these recommendations and records an explicit decision: Accept, Edit, or Reject. Only after an approved remediation is applied manually in the simulated lab and validated through follow-up connectivity tests is the troubleshooting incident concluded. The complete modular architecture of NetSage AI is illustrated in Figure 1.

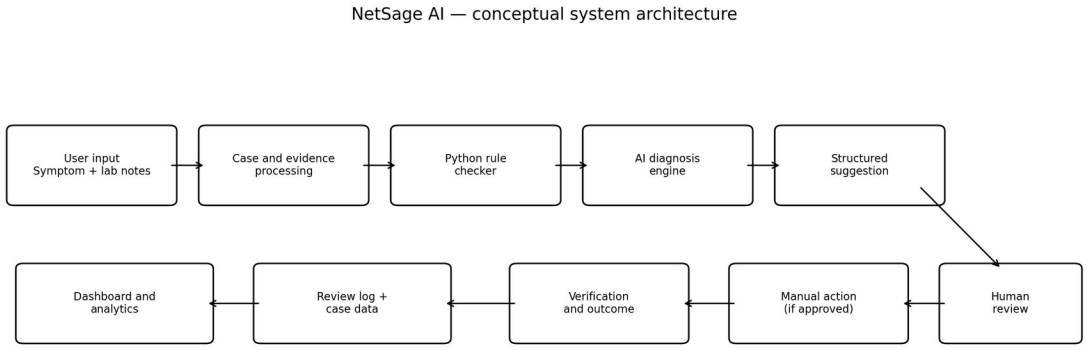


Figure 1: Conceptual Architecture of the NetSage AI Diagnostic System *(Design illustration depicting user input, deterministic checks, AI reasoning, and human review loop).*

2.1. Description of Tasks and Assignments

2.1.1. Requirement Analysis

The primary design requirement entails defining a rigorous data ingestion and structured output framework. Because informal user symptom statements are frequently ambiguous or misleading, the system requires structured input fields including device roles, IP subnet

schemes, VLAN allocations, network topology descriptions, and verbatim Cisco CLI command outputs. Crucially, the diagnostic engine must explicitly detect and report missing telemetry rather than making unwarranted assumptions regarding unobserved device states.

Furthermore, diagnostic outputs must conform to a standardized schema rather than unstructured narrative prose. Structured responses facilitate cognitive clarity, enabling the user to immediately distinguish established facts from speculative hypotheses, identify the next required diagnostic command, and audit the reasoning chain. Finally, the system mandates an explicit verification step before any fault can be categorized as resolved.

2.1.2. Case Dataset

To establish a repeatable basis for training and empirical evaluation, a structured case repository was architected. The dataset comprises 30 diverse enterprise network troubleshooting scenarios formatted in a tabular CSV schema. Standard fields include Case ID, Incident Title, Network Category, Severity Level, Reported Symptom, Topology Notes, CLI Command Telemetry, Ground-Truth Fault Label (for benchmarking), and Verification Criteria. The Case Explorer interface for browsing simulated networking incidents is depicted in Figure 2.

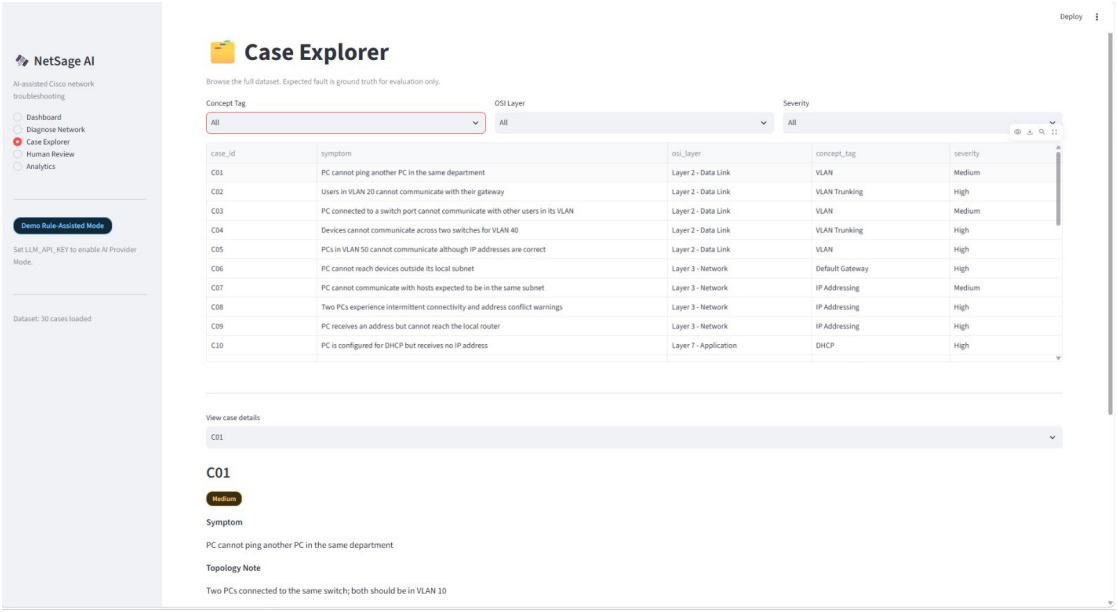


Figure 2: NetSage AI Case Explorer Telemetry Inspection Interface *(User interface displaying case categorization, OSI layer mapping, and severity levels).*

To maintain rigorous scientific validity during evaluation, ground-truth fault labels are strictly sequestered from the diagnostic prompt. Exposing evaluation labels to the AI inference engine would introduce data leakage and artificially inflate diagnostic accuracy.

The case library was subjected to data sanitization to eliminate duplicates, resolve ambiguous evidence, and ensure each case presents sufficient telemetry to support a unique technical root cause.

2.1.3. Supported Troubleshooting Categories

The diagnostic domain of NetSage AI addresses eight fundamental networking areas commonly encountered in enterprise switching and routing environments [1], [3]. The primary troubleshooting domains and their associated diagnostic evidence are summarized in Table 1.

Table 1: Supported Network Troubleshooting Categories and Diagnostic Evidence

Category	Typical Diagnostic Question	Diagnostic Telemetry / Evidence
VLAN	Is the endpoint assigned to the correct VLAN?	VLAN membership table, switchport access mode (`show vlan brief`)
Default Gateway	Is the host configured with the correct gateway IP?	Host IP configuration, gateway reachability (`ping`, `show ip interface brief`)
DHCP	Is the host leasing expected addressing parameters?	DHCP pool bindings, lease exhaustion, relay agents (`show ip dhcp binding`)
DNS	Does name resolution fail while IP connectivity succeeds?	Host resolver settings, local domain records, DNS server reachability
Routing	Does an active route exist toward the destination network?	Routing information base (`show ip route`), next-hop status, interface flags
ACL	Is security policy filtering legitimate network traffic?	Access-list rules, permit/deny statements, packet hit counters (`show access-lists`)
NAT	Is IP address translation required and correctly bound?	NAT translation table (`show ip nat translations`), inside/outside interfaces
Wireless	Is the client properly associated and authenticated?	SSID matching, WPA2/WPA3 pre-shared keys, wireless client association table

(Source: Cisco IOS Diagnostic and Configuration Reference [2])

These categories define the target coverage of the NetSage AI diagnostic framework. Automated diagnostic capabilities must only be claimed for categories corroborated by active deterministic parsing rules and verified experimental test runs.

2.1.4. Deterministic Python Rule Checker

The deterministic rule-checking engine utilizes Python pattern matching and regular expressions to evaluate telemetry without relying on probabilistic model inference [10]. The

rule checker operates upon verified network boundary conditions, including duplicate host IP assignments, mismatched subnet masks between communicating peers, default gateways outside the local subnet range, administratively down interface states, missing VLAN IDs on 802.1Q trunks, and unroutable destination prefixes.

For example, when CLI telemetry reveals an interface status of `administratively down`, the rule checker flags the interface as a physical-layer fault requiring a `no shutdown` command. Similarly, bitwise subnet calculations determine whether an endpoint's configured default gateway falls within its local network segment. Deterministic checks provide instantaneous, mathematically rigorous boundary validation that serves as an invariant foundation for subsequent AI reasoning.

2.1.5. AI Diagnosis Engine

The AI diagnosis engine processes multi-source case data, combining user-reported symptoms, topology notes, command output, and deterministic rule findings [5]. By employing structured system prompts, the model is constrained to generate valid JSON objects containing predefined fields: `root_cause`, `osi_layer`, `confidence`, `evidence`, `next_command`, `fix_steps`, and `reasoning_summary`.

Diagnostic confidence is categorized into High, Medium, or Low based on the availability and unambiguity of the evidence. These labels communicate heuristic certainty rather than calibrated statistical probabilities. When the available telemetry is incomplete, the engine is explicitly instructed to communicate diagnostic uncertainty and recommend a targeted CLI command to isolate the fault. The engine is strictly constrained from fabricating command output, hallucinating unseen configurations, or assuming an unverified hypothesis has resolved the incident [11].

2.1.6. Human Review Workflow

Human review constitutes a non-negotiable safety and governance mechanism within NetSage AI. The human reviewer scrutinizes the AI-generated hypothesis, examines the cited CLI evidence, and records one of three formal outcomes:

- Accept: The reviewer confirms that the hypothesis is rigorously supported by the evidence and approves the recommended corrective actions.
- Edit: The reviewer modifies the diagnosis, refines the cited evidence, or adjusts the remediation steps before proceeding.

- Reject: The reviewer determines that the hypothesis is technically erroneous, incomplete, or unsupported by the telemetry.

The system stores the initial AI suggestion and the final human-reviewed diagnosis as distinct records in the audit log. Preserving both versions allows instructors and researchers to evaluate model fidelity, track common failure modes, and analyze human intervention patterns over time.

2.1.7. Dashboard and Operational Analytics

An interactive graphical dashboard provides visibility into case library metrics, historical review trends, and model performance. Key metrics visualized include total cases, category distribution, severity ratings, human review status counts (accepted, edited, rejected), and the aggregate AI-human agreement rate. The real-time analytics dashboard displaying case metrics and reviewer agreements is illustrated in Figure 3.



Figure 3: Operational Analytics Dashboard and Diagnostic Distribution *(Dashboard interface presenting total cases, severity distributions, and AI-human agreement rates).*

All analytical calculations enforce rigorous mathematical definitions. For example, agreement rates are computed exclusively over fully reviewed cases with non-zero denominators. Missing data and malformed records are programmatically isolated to prevent skewed visualization totals.

2.1.8. Responsible AI Considerations

NetSage AI is architected around foundational responsible AI principles: empirical grounding, transparency of reasoning, explicit uncertainty communication, human oversight,

and fail-safe operation [5], [11]. The system clearly distinguishes observed empirical facts from inferential hypotheses. Automated configuration changes are disabled by design to eliminate the risk of accidental network disruption.

Security and privacy safeguards are systematically enforced. External API tokens and credentials must be secured within protected environment variables or local key vaults, never hardcoded into source repositories or saved within case records. Diagnostic telemetry logs are sanitized to exclude private network credentials, passwords, and sensitive architectural identifiers.

2.1.9. End-to-End Workflow

The end-to-end operational diagnostic process comprises eleven sequential procedural stages:

1. Select a predefined scenario from the case repository or input a new network symptom.
2. Input relevant Cisco CLI command outputs, device topology notes, and addressing schemes.
3. Validate, sanitize, and structure the ingested diagnostic telemetry.
4. Execute deterministic Python rule checks to evaluate boundary conditions and known failure signatures.
5. Generate a structured AI diagnostic hypothesis utilizing the configured inference engine.
6. Display the proposed root cause, OSI layer, confidence indicator, evidence citations, next command, and remediation steps.
7. Engage the human reviewer to evaluate the hypothesis and record an Accept, Edit, or Reject decision.
8. Upon approval, manually apply the recommended configuration remediation in the simulated network laboratory.
9. Execute follow-up verification tests (e.g., ping, traceroute, state checks) to validate end-to-end service restoration.
10. Persist the complete incident record, including original AI output, human review notes, and verification results, to the audit log.
11. Aggregate review history to update dashboard analytics and longitudinal agreement statistics.

The complete step-by-step diagnostic and review workflow is illustrated in Figure 4.

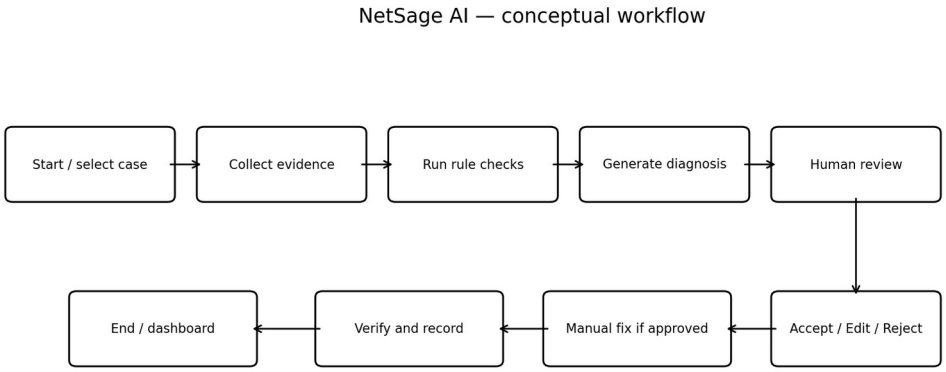


Figure 4: End-to-End Diagnostic and Human-in-the-Loop Review Workflow *(Procedural flow diagram from incident selection through rule evaluation, AI diagnosis, human review, and manual verification).*

This sequential procedure guarantees that every technical diagnosis is systematically validated against empirical evidence and subject to human oversight before implementation.

2.2. Tools, Technologies and Methods Used

The NetSage AI architecture leverages a modular software engineering methodology that decouples the presentation layer, data ingestion pipeline, deterministic rule checker, AI inference engine, and persistent review storage. The architectural components, technologies, and implementation tools utilized across the NetSage AI system are detailed in Table 2

Table 2: Tools, Technologies, and Engineering Methods Used

Technology / Tool	Architectural Role in NetSage AI	Validation & Verification Method
Cisco Packet Tracer	Network device, link, VLAN, and routing simulation environment	Verification of laboratory topologies and scenario configurations [4]
Python 3.12	Core application logic, deterministic rule checks, and data pipelines	Modular unit testing and automated assertion test suites [10]
Streamlit Framework	Interactive, browser-based graphical user interface and forms	UI component lifecycle testing and session state validation [6]
Pandas Library	Tabular data ingestion, case filtering, and telemetry processing	Data schema enforcement and missing value handling [12]
Plotly Graphing	Interactive telemetry visualization and categorical distribution charts	Visual rendering audits and metric calculation checks [7]
CSV Storage	Persistent tabular storage for case libraries and human review logs	File integrity validation and audit trail schema checks

Markdown Prompting	Structured prompt engineering for deterministic model outputs	JSON schema validation and adherence checks [5]
AI / LLM API	Contextual telemetry interpretation and explainable reasoning	Evaluation against ground-truth case benchmarks [13], [14]
Environment Vars	Secure storage for API keys and operational secrets	Credential isolation audits and environment checks

By enforcing strict architectural separation of concerns, the system permits deterministic rule checks to be evaluated, debugged, and versioned independently of the probabilistic language model components.

2.3. Challenges Faced and Solutions Provided

Engineering an AI-assisted diagnostic system for computer networking introduces significant challenges concerning input ambiguity, model hallucination, and operational safety. A systematic matrix of potential challenges and corresponding architectural safeguards is presented in Table 3.

Table 3: Potential System Challenges and Recommended Handling Approaches

Potential Challenge	Technical Consequence	Recommended Mitigation Strategy
Incomplete CLI Telemetry	Diagnosis becomes underdetermined and speculative	Detect missing evidence and prompt for targeted follow-up commands
Multiple Plausible Causes	Single symptom corresponds to several valid faults	Present differential diagnoses and rank hypotheses by evidence strength
Unsupported AI Assertions	Model asserts unobserved configuration states (hallucination)	Constrain model to cite verbatim CLI evidence; enforce schema validation
Inconsistent CLI Formats	String parsing failures across different IOS versions	Implement robust regex patterns and fallback exception handlers
Ground-Truth Data Leakage	Evaluation labels bias diagnostic prompts	Strictly isolate benchmarking labels from the runtime diagnostic context
Unavailable API Credentials	Live model inference fails or times out	Provide graceful fallback to deterministic demonstration mode
Malformed Schema Output	Missing required fields causes application crashes	Apply Pydantic/JSON schema validation with retry mechanisms
Review History Inconsistency	Confusion between original AI output and human edits	Persist separate immutable fields for AI suggestions and human revisions
Unsafe Configuration Changes	Erroneous automated fix disrupts active network topology	Enforce human-in-the-loop review and require manual lab application

Distorted Dashboard Metrics	Incorrect denominators produce misleading success rates	Standardize metric definitions and validate against sample records

In future production deployments, each of these potential challenges should be continuously benchmarked against real-world user interaction logs to monitor mitigation effectiveness.

2.4. Outcome of Internship / Certification

The project specification outlines key engineering deliverables: an operational case repository, deterministic rule checkers, a structured prompt library, human review interfaces, and an analytics dashboard. In accordance with rigorous scientific standards, deliverables must be categorized across four developmental stages: 'Implemented and Verified', 'Implemented Pending Verification', 'Planned Architecture', and 'Future Research Scope'. Project milestones must reflect verified empirical evidence rather than theoretical requirements.

3. Learning and Observations

The development of NetSage AI provided an integrative learning experience uniting enterprise networking principles, deterministic software engineering, prompt engineering, and responsible AI governance. The primary educational takeaway is the recognition that technical troubleshooting must proceed as an evidence-driven scientific process rather than intuitive guesswork. Furthermore, the work underscores the vital distinction between theoretical design specifications and empirically validated software performance.

3.1. Technical Skills Acquired

Networking Fundamentals

The project reinforced core networking paradigms across Layers 1 through 7 of the OSI model [1], [3]. Key concepts investigated include:

- IP Addressing & Subnetting: Bitwise subnet mask operations determine local vs. remote network destinations.
- Default Gateways: Gateways serve as essential layer-3 egress points for inter-subnet communication.
- VLAN Segmentation: VLANs partition Layer 2 broadcast domains; mismatched assignments isolate hosts.

- 802.1Q Trunking: Trunk links multiplex traffic across switches; prune lists restrict VLAN propagation.
- Routing Tables: Routing protocols populate the RIB/FIB; missing routes result in immediate packet drops.
- DHCP Services: Automated dynamic lease assignment of IP, mask, gateway, and DNS parameters.
- DNS Resolution: Domain name translation; DNS lookup failure mimics total network reachability loss.
- Access Control Lists: Packet filtering based on IP/port criteria; rule ordering determines policy outcome.
- Network Address Translation: Mapping private RFC 1918 addresses to public routable IP addresses.
- Wireless Networking: 802.11 SSID association, encryption negotiation, and upstream bridge validation.

Cisco Command Interpretation

Interpreting Cisco IOS CLI command output requires analyzing parameters in their broader topological context [2]. A breakdown of fundamental Cisco IOS verification commands utilized for telemetry extraction is presented in Table 4.

Table 4: Key Cisco IOS Diagnostic Commands and Inspection Roles

Cisco Command	Telemetry Inspection Focus	Diagnostic Troubleshooting Role
`show ip interface brief`	Interface operational status and IP address bindings	Verify whether interfaces are up/up, down/down, or administratively down
`show ip route`	Routing information base, next-hops, and default gateway	Determine if an active route exists toward the destination IP prefix
`show vlan brief`	Active VLAN IDs and access-port interface memberships	Confirm whether host access ports are assigned to expected broadcast domains
`show interfaces trunk`	802.1Q trunking encapsulation, native VLAN, allowed list	Detect whether trunk links are forwarding or blocking required VLAN traffic
`show access-lists`	Configured ACEs, permit/deny rules, and packet hit counters	Identify whether security policies are discarding legitimate client traffic

(Source: Cisco CCNA Official Certification Guide [3])

Crucially, CLI command telemetry must be interpreted holistically. The presence of a valid routing entry does not guarantee end-to-end transport if an intermediate trunk blocks the packet's VLAN or an ACL denies the return traffic. Comprehensive verification requires multi-stage telemetry cross-referencing.

Python Development

Building the deterministic rule-checking engine provided intensive experience in developing modular Python code, structured regex parsers, and robust error handling routines [10]. Deterministic functions were engineered around clean input contracts, unambiguous logical assertions, and predictable return schemas. Exception handling mechanisms ensure graceful recovery when users submit incomplete or malformed CLI outputs.

AI Prompt Design

Prompt engineering for technical troubleshooting requires rigorous constraints [5]. The NetSage AI prompts mandate JSON schema adherence, require verbatim citations of supplied CLI evidence, enforce explicit uncertainty declarations, and prohibit assertions claiming configuration changes have already been applied. Response validation pipelines verify model outputs against formal schemas prior to user presentation.

Data Handling and Visualization

Managing the case repository using Pandas provided practical training in tabular data manipulation, validation, and transformation [12]. Metric calculation routines were engineered to eliminate denominator biases. Interactive charts rendered via Plotly ensure that telemetry distributions and agreement percentages communicate accurate empirical findings [7].

Testing and Debugging

Developing test cases encompassing standard inputs, boundary conditions, malformed outputs, and conflicting evidence reinforced rigorous quality assurance practices. The project demonstrated that a planned or conceptual test is fundamentally distinct from an executed test backed by empirical evidence.

3.2. Professional / Soft Skills Developed

Beyond technical competencies, the project cultivated critical professional capabilities. Analytical problem-solving was honed by evaluating complex, multi-cause network failures and resisting premature conclusions. Technical documentation skills were strengthened by

producing precise, transparent specifications detailing architectural assumptions, boundary limitations, and empirical methods.

The project also advanced ethical awareness regarding responsible AI deployment. The human-in-the-loop workflow instilled a disciplined mindset prioritizing human oversight, auditability, and safety over unconstrained automation. Project management skills were developed by organizing interconnected modules—data schemas, rule checkers, prompt templates, review logs, and analytics—into a cohesive engineering roadmap.

4. Results and Discussions

The evaluation framework of NetSage AI is designed to establish whether the system satisfies its core technical objectives under controlled laboratory conditions [8]. To prevent the publication of speculative performance claims, this chapter details the formal evaluation methodology and analytical metrics framework while clearly identifying unverified experimental criteria.

4.1. Key Findings or Results

The system must be evaluated against reproducible empirical benchmarks. Each acceptance target requires an operational test procedure, predefined expected behavior, recorded actual outcome, and verifiable telemetry logs. The recommended test procedure comprises eight verification stages:

1. Verify application initialization and dependency resolution across the specified runtime environment.
2. Validate dataset ingestion and ensure graceful error handling for missing or malformed records.
3. Execute baseline diagnostic tests on known troubleshooting cases and log generated outputs.
4. Evaluate each deterministic Python rule against positive and negative test controls.
5. Audit AI diagnostic outputs for verbatim evidence citations and appropriate uncertainty communication.
6. Test Accept, Edit, and Reject review workflows, confirming separate storage of original and reviewed diagnoses.
7. Reconcile dashboard aggregate totals against direct database and CSV record counts.
8. Execute end-to-end Packet Tracer simulation scenarios, documenting before-and-after verification evidence.

4.2. Data Analysis

Empirical evaluation relies upon systematically analyzing case distributions, review statuses, and agreement metrics. The analytical framework for evaluating system diagnostic efficacy and reviewer interaction is summarized in Table 5.

Table 5: Data Analysis, Performance Metrics, and Review Evaluation Framework

Analysis Metric	Mathematical Formulation / Method	Analytical Interpretation Caution
Cases by Category	Count records grouped by network category	Validate against inconsistent or overlapping category labels
Cases by Severity	Count records grouped by severity ranking	Enforce standardized severity definitions (Low, Medium, High)
Review Status Count	Count records grouped by Accept, Edit, Reject, Pending	Exclude unreviewed / pending records from performance totals
Human Correction Rate	(Edited + Rejected Diagnoses) / Total Reviewed Diagnoses	Clearly define whether minor editorial changes constitute corrections
AI-Human Agreement	Accepted Diagnoses / Total Reviewed Diagnoses * 100	Reflects human reviewer concurrence; does not prove ground truth

The AI-human agreement proxy is formalized through the following mathematical expression:

Equation (1): AI-Human Agreement Proxy (%) = (Accepted Diagnoses / Total Reviewed Diagnoses) * 100

This formula must be computed strictly when the denominator (Total Reviewed Diagnoses) is greater than zero and derived from authentic review records. This agreement rate measures human reviewer consensus under the defined evaluation protocol. It should not be conflated with objective ground-truth accuracy, which requires formal validation against verified laboratory fault injection benchmarks.

4.3. Interpretation of Results

When analyzing experimental diagnostic results, evaluators must look beyond whether the system merely generates a plausible response. High-quality diagnostic guidance must accurately identify the relevant OSI layer, cite concrete command evidence, recommend a useful next command, and convey uncertainty when multiple root causes remain viable. A diagnosis that sounds confident but lacks empirical support impairs technical learning.

Furthermore, human reviewer feedback provides invaluable diagnostic telemetry. Tracking why human operators edit or reject specific AI hypotheses exposes edge cases, command parsing gaps, and reasoning limitations. Finally, a plausible diagnosis is not equivalent to a

successful repair; incident resolution requires post-remediation verification through positive connectivity tests.

4.4. Contribution to the Organization / Academic Work

NetSage AI provides meaningful academic and educational contributions. It demonstrates a practical paradigm for combining deterministic Python validation with generative AI inference, offering a blueprint for educational troubleshooting assistants. The curated case library serves as a reusable pedagogical resource for enterprise networking education. Furthermore, the human-in-the-loop review workflow offers an extensible framework for studying human-AI collaboration in technical domains.

Additionally, the project exemplifies responsible AI principles in engineering education by emphasizing evidence grounding, explicit uncertainty communication, credential protection, and human sovereignty over network changes. Commercial deployment or organizational efficiency claims are omitted until verified by longitudinal field trials.

5. Conclusion

NetSage AI has been specified and architected as an intelligent troubleshooting assistant for Cisco networking laboratories. It addresses a fundamental educational challenge: bridging the divide between knowing individual CLI commands and synthesizing command evidence to diagnose complex network failures. The architecture achieves this by uniting structured case telemetry, deterministic Python rule verification, structured AI reasoning, and an authoritative human-in-the-loop review mechanism.

The deterministic rule checker and AI diagnosis engine fulfill distinct, complementary roles. The rule checker validates invariant syntax and configuration boundaries, while the AI engine analyzes holistic context and generates structured, explainable hypotheses. A foundational tenet of the design is that AI-generated hypotheses must never be treated as verified ground truth. Configurations must not be altered automatically. Preserving both initial AI proposals and final human review decisions ensures complete transparency, auditability, and educational value.

This research proposal establishes the architectural framework, operational workflows, toolchain integrations, and evaluation methodologies for NetSage AI. By following these rigorous guidelines, the project provides a solid, responsible foundation for future research and laboratory implementation.

5.1. Future Enhancements

- Advanced CLI Parsers: Enhance regular expression parsers to support diverse Cisco IOS-XE, NX-OS, and multi-vendor switch outputs.
- Expanded Case Library: Scale the case repository with complex multi-area OSPF, BGP, and spanning-tree failure scenarios.
- Automated Rule Testing: Implement continuous integration suites to test deterministic rules against regression test datasets.
- Exportable Case Reports: Develop automated reporting utilities to generate PDF audit summaries of completed troubleshooting sessions.
- Fine-Grained Evidence Mapping: Enhance UI visualizations to explicitly link each diagnostic claim to its corresponding CLI snippet.
- Ground-Truth Benchmarking: Conduct formal empirical accuracy evaluations comparing AI diagnoses against verified physical lab faults.

- Direct Lab Telemetry Integration: Explore secure, read-only API connectors to ingest live telemetry from Cisco Packet Tracer and GNS3.
- Interactive Follow-Up Dialog: Implement multi-turn conversational capabilities allowing users to submit follow-up command results.
- Enhanced Secrets Management: Integrate enterprise secret management services (e.g., HashiCorp Vault) for production credentials.
- Versioned Review Auditing: Implement full cryptographic versioning for diagnostic proposals and human remediation logs.

6. References

The following references represent official technical documentation, peer-reviewed scientific literature, and academic guidelines cited sequentially throughout this research proposal. All citations conform to uniform IEEE bibliographical standards [8].

[1] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Boston, MA, USA: Pearson Education, 2021.

[2] Cisco Systems, Inc., "Cisco IOS Master Command List and IP Routing Configuration Guide," Cisco Systems Documentation Portal, San Jose, CA, USA, 2024. [Online]. Available: https://www.cisco.com/c/en/us/support/index.html

[3] W. Odom, CCNA 200-301 Official Cert Guide, Volume 1 and Volume 2, Indianapolis, IN, USA: Cisco Press, Pearson Education, 2020.

[4] Cisco Networking Academy, "Cisco Packet Tracer: Network Simulation and Visualization Software," Cisco Systems, Inc., San Jose, CA, USA, 2024. [Online]. Available: https://www.netacad.com/

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 24824–24837, Dec. 2022.

[6] Streamlit Inc., "Streamlit Documentation: Rapid Application Development for Data and Machine Learning," Snowflake Inc., San Mateo, CA, USA, 2024. [Online]. Available: https://docs.streamlit.io/

[7] Plotly Technologies Inc., "Plotly Python Open Source Graphing Library Reference," Plotly Technologies Inc., Montreal, QC, Canada, 2024. [Online]. Available: https://plotly.com/python/

[8] MIT Academy of Engineering, "Guidelines for Student Research Proposals and Project Work (Academic Year 2026-27)," Department of Computer Engineering, Alandi (D), Pune, Maharashtra, India, 2026.

[9] All India Council for Technical Education (AICTE), "National Internship Policy: Guidelines and Operational Procedures for Technical Institutions," Ministry of Education, Government of India, New Delhi, India, 2023.

[10] Python Software Foundation, "Python 3.12 Reference Manual and Standard Library," Python Software Foundation, Wilmington, DE, USA, 2024. [Online]. Available: https://docs.python.org/3/

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, Dec. 2020.

[12] W. McKinney and the pandas development team, "pandas: Powerful Python Data Analysis Toolkit," Version 2.2, PyData Organization, 2024. [Online]. Available: https://pandas.pydata.org/docs/

[13] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Foundation Models," Google Research Technical Report, Mountain View, CA, USA, Dec. 2023. [Online]. Available: https://blog.google/technology/ai/google-gemini-ai/

[14] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv preprint arXiv:2307.09288, Jul. 2023.

APPENDIX E: PLAGIARISM / SIMILARITY REPORT

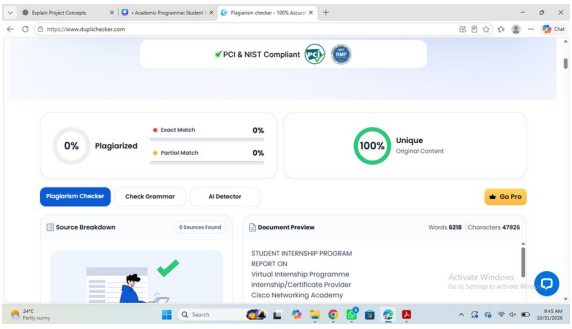
Report Details

Particulars	Details
Title of Report	Virtual Internship Programme – Student Internship Program (SIP) Report
Name of Student	Kadam Om Shivnath
PRN / Exam Seat No.	202501040064 / 41
Internship Provider / Duration	Cisco Networking Academy / 8 weeks
Department	Department of Computer Engineering
Institute	MIT Academy of Engineering, Alandi (D), Pune – 412105
Internship Guide	Prof. Disha Sengupta
Academic Year	2026-27

Similarity Check Summary

Particulars	Details
Plagiarism / Similarity Checking Tool	DupliChecker (www.duplichecker.com)
Date of Check	31/10/2026
Plagiarized Content	0 %
Exact Match / Partial Match	0 % / 0 %
Unique / Original Content	100 %
Sources Found	0
Words Checked	6218

Similarity Check Result



Screenshot of the DupliChecker similarity check result (31/10/2026)

Declaration

I hereby declare that this report is the result of my own work carried out during the Student Internship Program. All sources of information, figures and technical material used have been duly acknowledged and cited in the References section. The similarity index reported above has been obtained by checking the final version of this report.

Student Signature	**Internship Guide**
Om Shivnath Kadam	Prof. Disha Sengupta